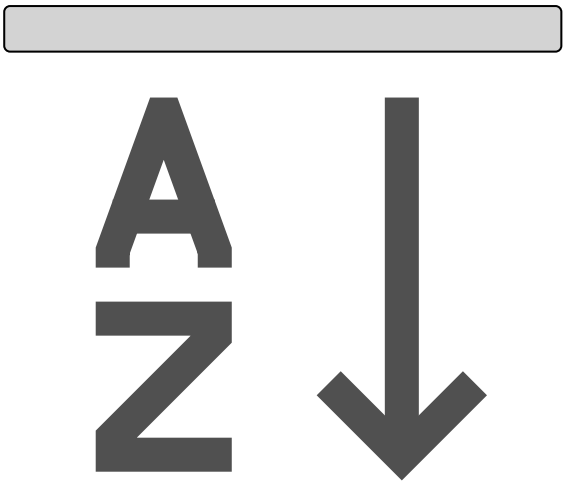
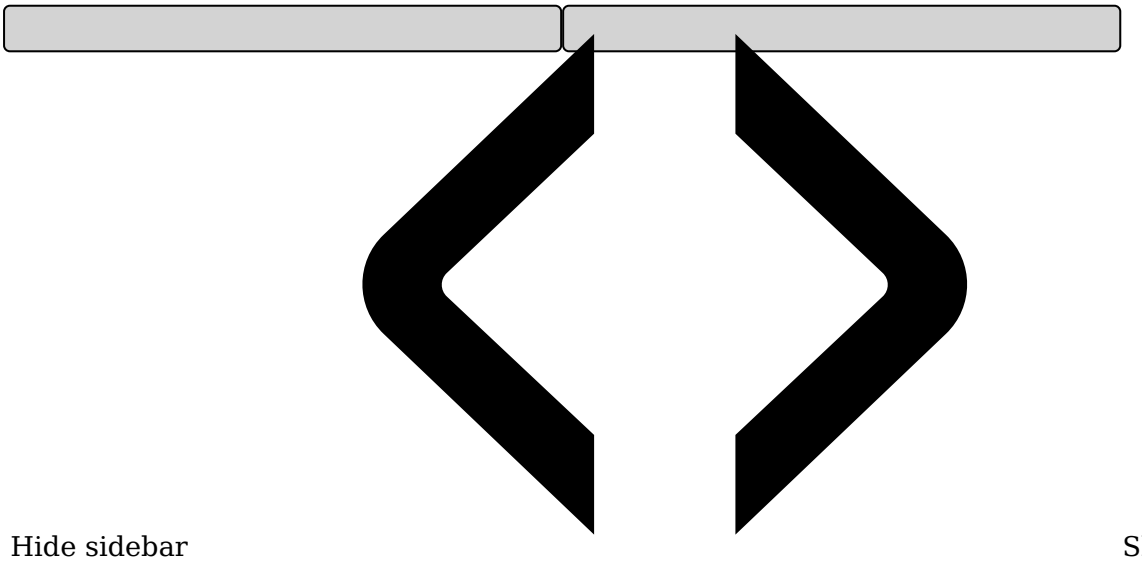
JavaScript is disabled on your browser.



1. Description
2. Method Summary
3. Method Details
    1. thenApply(Function)
    2. thenApplyAsync(Function)
    3. thenApplyAsync(Function, Executor)
    4. thenAccept(Consumer)
    5. thenAcceptAsync(Consumer)
    6. thenAcceptAsync(Consumer, Executor)
    7. thenRun(Runnable)
    8. thenRunAsync(Runnable)
    9. thenRunAsync(Runnable, Executor)
    10. thenCombine(CompletionStage, BiFunction)
    11. thenCombineAsync(CompletionStage, BiFunction)
    12. thenCombineAsync(CompletionStage, BiFunction, Executor)
    13. thenAcceptBoth(CompletionStage, BiConsumer)
    14. thenAcceptBothAsync(CompletionStage, BiConsumer)
    15. thenAcceptBothAsync(CompletionStage, BiConsumer, Executor)
    16. runAfterBoth(CompletionStage, Runnable)
    17. runAfterBothAsync(CompletionStage, Runnable)
    18. runAfterBothAsync(CompletionStage, Runnable, Executor)
    19. applyToEither(CompletionStage, Function)
    20. applyToEitherAsync(CompletionStage, Function)
    21. applyToEitherAsync(CompletionStage, Function, Executor)
    22. acceptEither(CompletionStage, Consumer)
    23. acceptEitherAsync(CompletionStage, Consumer)
    24. acceptEitherAsync(CompletionStage, Consumer, Executor)
    25. runAfterEither(CompletionStage, Runnable)
    26. runAfterEitherAsync(CompletionStage, Runnable)
    27. runAfterEitherAsync(CompletionStage, Runnable, Executor)
    28. thenCompose(Function)
    29. thenComposeAsync(Function)
    30. thenComposeAsync(Function, Executor)
    31. handle(BiFunction)
    32. handleAsync(BiFunction)
    33. handleAsync(BiFunction, Executor)

34. whenComplete(BiConsumer)
35. whenCompleteAsync(BiConsumer)
36. whenCompleteAsync(BiConsumer, Executor)
37. exceptionally(Function)
38. exceptionallyAsync(Function)
39. exceptionallyAsync(Function, Executor)
40. exceptionallyCompose(Function)
41. exceptionallyComposeAsync(Function)
42. exceptionallyComposeAsync(Function, Executor)
43. toCompletableFuture()



Interface CompletionStage<T>

Type Parameters:
T - the type of values the stage produces or consumes

All Known Implementing Classes:
CompletableFuture

---

public interface CompletionStage<T>
A stage of a possibly asynchronous computation, that performs an action or computes a value when another CompletionStage completes. A stage completes upon termination of its computation, which may in turn trigger other dependent stages. The functionality defined in this interface takes only a few basic forms, which expand out to a larger set of methods to capture a range of usage styles:

- The computation performed by a stage may be expressed as a Function, Consumer, or Runnable (using methods with names including *apply*, *accept*, or *run*, respectively) depending on whether it requires arguments and/or produces results. For example:

```
stage.thenApply(x -> square(x))
     .thenAccept(x -> System.out.print(x))
     .thenRun(() -> System.out.println());
```

An additional form (*compose*) allows the construction of computation pipelines from functions returning completion stages.

Any argument to a stage's computation is the outcome of a triggering stage's computation.

- One stage's execution may be triggered by completion of a single stage, or both of two stages, or either of two stages. Dependencies on a single stage are arranged using methods with prefix *then*. Those triggered by completion of *both* of two stages may *combine* their results or effects, using correspondingly named methods. Those triggered by *either* of two stages make no guarantees about which of the results or effects are used for the dependent stage's computation.
- Dependencies among stages control the triggering of computations, but do not otherwise guarantee any particular ordering. Additionally, execution of a new stage's computations may be arranged in any of three ways: default execution, default asynchronous execution (using methods with suffix *async* that employ the stage's default asynchronous execution facility), or custom (via a supplied `Executor`). The execution properties of default and async modes are specified by CompletionStage implementations, not this interface. Methods with explicit Executor arguments may have arbitrary execution properties, and might not even support concurrent execution, but are arranged for processing in a way that accommodates asynchrony.
- Two method forms (`handle` and `whenComplete`) support unconditional computation whether the triggering stage completed normally or exceptionally. Method `exceptionally` supports computation only when the triggering stage completes exceptionally, computing a replacement result, similarly to the java `catch` keyword. In all other cases, if a stage's computation terminates abruptly with an (unchecked) exception or error, then all dependent stages requiring its completion complete exceptionally as well, with a `CompletionException` holding the exception as its cause. This convention distinguishes exceptions in an action itself from those it depends on. If they are to be handled in the same way, instead catch `RuntimeException` (possibly inspecting the exception's `getCause()`). If a stage is dependent on *both* of two stages, and both complete exceptionally, then the CompletionException may correspond
to either one of these exceptions. If a stage is dependent on *either* of two others, and only one of them completes exceptionally, no guarantees are made about whether the dependent stage completes normally or exceptionally. In the case of method `whenComplete`, when the supplied action itself encounters an exception, then the stage completes exceptionally with that exception unless the source stage also completed exceptionally, in which case the exceptional completion from the source stage is given preference and propagated to the dependent stage. Applications are encouraged to maintain these conventions, avoiding unnecessary nesting when rethrowing, as in `throw (ex instanceof CompletionException) ? ex : new CompletionException(ex)`.

All methods adhere to the above triggering, execution, and exceptional completion specifications (which are not repeated in individual method specifications). Additionally, while arguments used to pass a completion result (that is, for parameters of type T) for methods accepting them may be null, passing a null value for any other parameter will result in a NullPointerException being thrown.

Method form handle is the most general way of creating a continuation stage, unconditionally performing a computation that is given both the result and exception (if any) of the triggering CompletionStage, and computing an arbitrary result. Method whenComplete is similar, but preserves the result of the triggering stage instead of computing a new one. Because a stage's normal result may be null, both methods should have a computation structured thus:

```
(result, exception) -> {
  if (exception == null) {
    // triggering stage completed normally
  } else {
    // triggering stage completed exceptionally
  }
}
```

The CompletionStage interface does not define methods for initially creating, forcibly completing normally or exceptionally, probing completion status or results, or awaiting completion of a stage. Implementations of CompletionStage may provide means of achieving such effects, as appropriate. Method toCompletableFuture() enables interoperability among different implementations of this interface by providing a common conversion type.

Memory consistency effects: Actions in a thread prior to the submission of a computation producing a CompletionStage *happen-before* that computation begins. And actions taken by CompletionStage x *happen-before* actions of any dependent stage subsequent to x's completion.

Since:
1.8

Method Summary

All Methods	Instance Methods
Abstract Methods	Default Methods

Modifier and Type
Method
Description
CompletionStage<Void>
acceptEither(CompletionStage<? extends T> other, Consumer<? super T> action)

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied action.

CompletionStage<Void> acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action)

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied action.

CompletionStage<Void> acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action, Executor executor)

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied action.

<U> CompletionStage<U> applyToEither(CompletionStage<? extends T> other, Function<? super T, U> fn)

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied function.

<U> CompletionStage<U> applyToEitherAsync(CompletionStage<? extends T> other, Function<? super T, U> fn)

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied function.

<U> CompletionStage<U> applyToEitherAsync(CompletionStage<? extends T> other, Function<? super T, U> fn, Executor executor)

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function.

CompletionStage<T> exceptionally(Function<Throwable, ? extends T> fn)

Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function.

default CompletionStage<T> exceptionallyAsync(Function<Throwable, ? extends T> fn)

Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function, using this stage's default asynchronous execution facility.

default CompletionStage<T> exceptionallyAsync(Function<Throwable, ? extends T> fn, Executor executor)

Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function, using the supplied Executor.

default CompletionStage<T>
exceptionallyCompose(Function<Throwable, ? extends
CompletionStage<T>> fn)
Returns a new CompletionStage that, when this stage completes exceptionally, is composed using the results of the supplied function applied to this stage's exception.
default CompletionStage<T>
exceptionallyComposeAsync(Function<Throwable, ? extends
CompletionStage<T>> fn)
Returns a new CompletionStage that, when this stage completes exceptionally, is composed using the results of the supplied function applied to this stage's exception, using this stage's default asynchronous execution facility.
default CompletionStage<T>
exceptionallyComposeAsync(Function<Throwable, ? extends
CompletionStage<T>> fn, Executor executor)
Returns a new CompletionStage that, when this stage completes exceptionally, is composed using the results of the supplied function applied to this stage's exception, using the supplied Executor.
<U> CompletionStage<U>
handle(BiFunction<? super T, Throwable, ? extends U> fn)
Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed with this stage's result and exception as arguments to the supplied function.
<U> CompletionStage<U>
handleAsync(BiFunction<? super T, Throwable, ? extends U> fn)
Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using this stage's default asynchronous execution facility, with this stage's result and exception as arguments to the supplied function.
<U> CompletionStage<U>
handleAsync(BiFunction<? super T, Throwable, ?
extends U> fn, Executor executor)
Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using the supplied executor, with this stage's result and exception as arguments to the supplied function.
CompletionStage<Void>
runAfterBoth(CompletionStage<?> other, Runnable action)
Returns a new CompletionStage that, when this and the other given stage both complete normally, executes the given action.
CompletionStage<Void>
runAfterBothAsync(CompletionStage<?> other, Runnable action)
Returns a new CompletionStage that, when this and the other given stage both complete normally, executes the given action using this stage's default asynchronous execution facility.
CompletionStage<Void>
runAfterBothAsync(CompletionStage<?> other, Runnable action,
Executor executor)
Returns a new CompletionStage that, when this and the other given stage both complete normally, executes the given action using the supplied executor.

CompletionStage<Void>
runAfterEither(CompletionStage<?> other, Runnable action)
Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action.
CompletionStage<Void>
runAfterEitherAsync(CompletionStage<?> other, Runnable action)
Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using this stage's default asynchronous execution facility.
CompletionStage<Void>
runAfterEitherAsync(CompletionStage<?> other, Runnable action, Executor executor)
Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using the supplied executor.
CompletionStage<Void>
thenAccept(Consumer<? super T> action)
Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied action.
CompletionStage<Void>
thenAcceptAsync(Consumer<? super T> action)
Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied action.
CompletionStage<Void>
thenAcceptAsync(Consumer<? super T> action, Executor executor)
Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied action.
<U> CompletionStage<Void>
thenAcceptBoth(CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)
Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied action.
<U> CompletionStage<Void>
thenAcceptBothAsync(CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)
Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied action.
<U> CompletionStage<Void>
thenAcceptBothAsync(CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action, Executor executor)
Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied action.

<U> CompletionStage<U>
thenApply(Function<? super T, ? extends U> fn)
Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied function.
<U> CompletionStage<U>
thenApplyAsync(Function<? super T, ? extends U> fn)
Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied function.
<U> CompletionStage<U>
thenApplyAsync(Function<? super T, ? extends U> fn, Executor executor)
Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied function.
<U,V> CompletionStage<V>
thenCombine(CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)
Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied function.
<U,V> CompletionStage<V>
thenCombineAsync(CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)
Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied function.
<U,V> CompletionStage<V>
thenCombineAsync(CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn, Executor executor)
Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function.
<U> CompletionStage<U>
thenCompose(Function<? super T, ? extends CompletionStage<U>> fn)
Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function.
<U> CompletionStage<U>
thenComposeAsync(Function<? super T, ? extends CompletionStage<U>> fn)
Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using this stage's default asynchronous execution facility.
<U> CompletionStage<U>
thenComposeAsync(Function<? super T, ? extends CompletionStage<U>> fn, Executor executor)

Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using the supplied Executor.

`CompletionStage<Void> thenRun(Runnable action)`
Returns a new CompletionStage that, when `this` stage completes normally, executes the given action.

`CompletionStage<Void> thenRunAsync(Runnable action)`
Returns a new CompletionStage that, when `this` stage completes normally, executes the given action using `this` stage's default asynchronous execution facility.

`CompletionStage<Void> thenRunAsync(Runnable action, Executor executor)`
Returns a new CompletionStage that, when `this` stage completes normally, executes the given action using the supplied Executor.

`CompletableFuture<T> toCompletableFuture()`
Returns a `CompletableFuture` maintaining the same completion properties as `this` stage.

`CompletionStage<T> whenComplete(BiConsumer<? super T, ? super Throwable> action)`
Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action when `this` stage completes.

`CompletionStage<T> whenCompleteAsync(BiConsumer<? super T, ? super Throwable> action)`
Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action using `this` stage's default asynchronous execution facility when `this` stage completes.

`CompletionStage<T> whenCompleteAsync(BiConsumer<? super T, ? super Throwable> action, Executor executor)`
Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action using the supplied Executor when `this` stage completes.

Method Details

thenApply

<U> CompletionStage<U> thenApply(Function<? super T, ? extends U> fn)
Returns a new CompletionStage that, when `this` stage completes normally, is executed with `this` stage's result as the argument to the supplied function.

This method is analogous to `Optional.map` and `Stream.map`.

See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:
`U` - the function's return type
Parameters:
`fn` - the function to use to compute the value of the returned CompletionStage
Returns:
the new CompletionStage

- ## thenApplyAsync

<U> CompletionStage<U> thenApplyAsync(Function<? super T, ? extends U> fn)
Returns a new CompletionStage that, when `this` stage completes normally, is executed using `this` stage's default asynchronous execution facility, with `this` stage's result as the argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.
Type Parameters:
`U` - the function's return type
Parameters:
`fn` - the function to use to compute the value of the returned CompletionStage
Returns:
the new CompletionStage

- ## thenApplyAsync

<U> CompletionStage<U> thenApplyAsync(Function<? super T, ? extends U> fn, Executor executor)
Returns a new CompletionStage that, when `this` stage completes normally, is executed using the supplied Executor, with `this` stage's result as the argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.
Type Parameters:
`U` - the function's return type
Parameters:
`fn` - the function to use to compute the value of the returned CompletionStage
`executor` - the executor to use for asynchronous execution
Returns:
the new CompletionStage

- ## thenAccept

CompletionStage<Void> thenAccept(Consumer<? super T> action)
Returns a new CompletionStage that, when `this` stage completes normally, is executed with `this` stage's result as the argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Parameters:
`action` - the action to perform before completing the returned CompletionStage
Returns:
the new CompletionStage

- ### thenAcceptAsync

  CompletionStage<Void> thenAcceptAsync(Consumer<? super T> action)
  Returns a new CompletionStage that, when `this` stage completes normally, is executed using `this` stage's default asynchronous execution facility, with `this` stage's result as the argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `action` - the action to perform before completing the returned CompletionStage
  Returns:
  the new CompletionStage

- ### thenAcceptAsync

  CompletionStage<Void> thenAcceptAsync(Consumer<? super T> action, Executor executor)
  Returns a new CompletionStage that, when `this` stage completes normally, is executed using the supplied Executor, with `this` stage's result as the argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `action` - the action to perform before completing the returned CompletionStage
  `executor` - the executor to use for asynchronous execution
  Returns:
  the new CompletionStage

- ### thenRun

  CompletionStage<Void> thenRun(Runnable action)
  Returns a new CompletionStage that, when `this` stage completes normally, executes the given action. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `action` - the action to perform before completing the returned CompletionStage
  Returns:
  the new CompletionStage

- ### thenRunAsync

  CompletionStage<Void> thenRunAsync(Runnable action)
  Returns a new CompletionStage that, when `this` stage completes normally, executes the given action using `this` stage's default asynchronous execution facility. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `action` - the action to perform before completing the returned CompletionStage
  Returns:
  the new CompletionStage

- ### thenRunAsync

  CompletionStage<Void> thenRunAsync(Runnable action, Executor executor)
  Returns a new CompletionStage that, when `this` stage completes normally, executes the given action using the supplied Executor. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `action` - the action to perform before completing the returned CompletionStage
  `executor` - the executor to use for asynchronous execution
  Returns:
  the new CompletionStage

- ### thenCombine

  <U,V> CompletionStage<V> thenCombine(CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed with the two results as arguments to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.
  Type Parameters:
  `U` - the type of the other CompletionStage's result
  `V` - the function's return type
  Parameters:
  `other` - the other CompletionStage
  `fn` - the function to use to compute the value of the returned CompletionStage
  Returns:
  the new CompletionStage

- ## thenCombineAsync

  <U,V> CompletionStage<V> thenCombineAsync (CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using `this` stage's default asynchronous execution facility, with the two results as arguments to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.
  Type Parameters:
  `U` - the type of the other CompletionStage's result
  `V` - the function's return type
  Parameters:
  `other` - the other CompletionStage
  `fn` - the function to use to compute the value of the returned CompletionStage
  Returns:
  the new CompletionStage
- ## thenCombineAsync

  <U,V> CompletionStage<V> thenCombineAsync (CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn, Executor executor)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.
  Type Parameters:
  `U` - the type of the other CompletionStage's result
  `V` - the function's return type
  Parameters:
  `other` - the other CompletionStage
  `fn` - the function to use to compute the value of the returned CompletionStage
  `executor` - the executor to use for asynchronous execution
  Returns:
  the new CompletionStage
- ## thenAcceptBoth

  <U> CompletionStage<Void> thenAcceptBoth (CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed with the two results as arguments to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:
`U` - the type of the other CompletionStage's result
Parameters:
`other` - the other CompletionStage
`action` - the action to perform before completing the returned CompletionStage
Returns:
the new CompletionStage

- ### thenAcceptBothAsync

<U> CompletionStage<Void> thenAcceptBothAsync (CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)
Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using `this` stage's default asynchronous execution facility, with the two results as arguments to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.
Type Parameters:
`U` - the type of the other CompletionStage's result
Parameters:
`other` - the other CompletionStage
`action` - the action to perform before completing the returned CompletionStage
Returns:
the new CompletionStage

- ### thenAcceptBothAsync

<U> CompletionStage<Void> thenAcceptBothAsync (CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action, Executor executor)
Returns a new CompletionStage that, when `this` and the other given stage both complete normally, is executed using the supplied executor, with the two results as arguments to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.
Type Parameters:
`U` - the type of the other CompletionStage's result
Parameters:
`other` - the other CompletionStage
`action` - the action to perform before completing the returned CompletionStage
`executor` - the executor to use for asynchronous execution
Returns:
the new CompletionStage

- ### runAfterBoth

  CompletionStage<Void> runAfterBoth(CompletionStage<?> other, Runnable action)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage
  Returns:
  the new CompletionStage

- ### runAfterBothAsync

  CompletionStage<Void> runAfterBothAsync(CompletionStage<?> other, Runnable action)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action using `this` stage's default asynchronous execution facility. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage
  Returns:
  the new CompletionStage

- ### runAfterBothAsync

  CompletionStage<Void> runAfterBothAsync(CompletionStage<?> other, Runnable action, Executor executor)
  Returns a new CompletionStage that, when `this` and the other given stage both complete normally, executes the given action using the supplied executor. See the `CompletionStage` documentation for rules covering exceptional completion.
  Parameters:
  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage
  `executor` - the executor to use for asynchronous execution
  Returns:
  the new CompletionStage

- ### applyToEither

  <U> CompletionStage<U> applyToEither(CompletionStage<? extends T> other, Function<? super T, U> fn)

Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed with the corresponding
result as argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:

`U` - the function's return type

Parameters:

`other` - the other CompletionStage

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

- ## applyToEitherAsync

  <U> CompletionStage<U> applyToEitherAsync (CompletionStage<? extends T> other, Function<? super T, U> fn)

  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using `this` stage's default asynchronous execution facility, with the corresponding
  result as argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

  Type Parameters:

  `U` - the function's return type

  Parameters:

  `other` - the other CompletionStage

  `fn` - the function to use to compute the value of the returned CompletionStage

  Returns:

  the new CompletionStage

- ## applyToEitherAsync

  <U> CompletionStage<U> applyToEitherAsync (CompletionStage<? extends T> other, Function<? super T, U> fn, Executor executor)

  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

  Type Parameters:

  `U` - the function's return type

  Parameters:

  `other` - the other CompletionStage

`fn` - the function to use to compute the value of the returned CompletionStage
`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

- ### acceptEither

  CompletionStage<Void> acceptEither(CompletionStage<? extends T> other, Consumer<? super T> action)
  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed with the corresponding result as argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

  Parameters:

  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage

  Returns:

  the new CompletionStage

- ### acceptEitherAsync

  CompletionStage<Void> acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action)
  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using `this` stage's default asynchronous execution facility, with the corresponding result as argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

  Parameters:

  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage

  Returns:

  the new CompletionStage

- ### acceptEitherAsync

  CompletionStage<Void> acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action, Executor executor)
  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied action. See the `CompletionStage` documentation for rules covering exceptional completion.

  Parameters:

  `other` - the other CompletionStage

`action` - the action to perform before completing the returned CompletionStage
`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

- ## runAfterEither

  CompletionStage<Void> runAfterEither(CompletionStage<?> other, Runnable action)

  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, executes the given action. See the `CompletionStage` documentation for rules covering exceptional completion.

  Parameters:

  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage

  Returns:

  the new CompletionStage

- ## runAfterEitherAsync

  CompletionStage<Void> runAfterEitherAsync(CompletionStage<?> other, Runnable action)

  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, executes the given action using `this` stage's default asynchronous execution facility. See the `CompletionStage` documentation for rules covering exceptional completion.

  Parameters:

  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage

  Returns:

  the new CompletionStage

- ## runAfterEitherAsync

  CompletionStage<Void> runAfterEitherAsync(CompletionStage<?> other, Runnable action, Executor executor)

  Returns a new CompletionStage that, when either `this` or the other given stage complete normally, executes the given action using the supplied executor. See the `CompletionStage` documentation for rules covering exceptional completion.

  Parameters:

  `other` - the other CompletionStage
  `action` - the action to perform before completing the returned CompletionStage
  `executor` - the executor to use for asynchronous execution

  Returns:

the new CompletionStage

- ## thenCompose

<U> CompletionStage<U> thenCompose(Function<? super T, ? extends CompletionStage<U>> fn)
Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function.

When `this` stage completes normally, the given function is invoked with `this` stage's result as the argument, returning another CompletionStage. When that stage completes normally, the CompletionStage returned is completed with the same value.

To ensure progress, the supplied function must arrange eventual completion of its result.

This method is analogous to `Optional.flatMap` and `Stream.flatMap`.

See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:
`U` - the type of the returned CompletionStage's result
Parameters:
`fn` - the function to use to compute another CompletionStage
Returns:
the new CompletionStage

- ## thenComposeAsync

<U> CompletionStage<U> thenComposeAsync(Function<? super T, ? extends CompletionStage<U>> fn)
Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using `this` stage's default asynchronous execution facility.

When `this` stage completes normally, the given function is invoked with `this` stage's result as the argument, returning another CompletionStage. When that stage completes normally, the CompletionStage returned is completed with the same value.

To ensure progress, the supplied function must arrange eventual completion of its result.

See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:
`U` - the type of the returned CompletionStage's result
Parameters:

`fn` - the function to use to compute another CompletionStage

Returns:

the new CompletionStage

- ## thenComposeAsync

<U> CompletionStage<U> thenComposeAsync(Function<? super T, ? extends CompletionStage<U>> fn, Executor executor)
Returns a new CompletionStage that is completed with the same value as the CompletionStage returned by the given function, executed using the supplied Executor.

When `this` stage completes normally, the given function is invoked with `this` stage's result as the argument, returning another CompletionStage. When that stage completes normally, the CompletionStage returned by `this` method is completed with the same value.

To ensure progress, the supplied function must arrange eventual completion of its result.

See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:

`U` - the type of the returned CompletionStage's result

Parameters:

`fn` - the function to use to compute another CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

- ## handle

<U> CompletionStage<U> handle(BiFunction<? super T, Throwable, ? extends U> fn)
Returns a new CompletionStage that, when `this` stage completes either normally or exceptionally, is executed with `this` stage's result and exception as arguments to the supplied function.

When `this` stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments, and the function's result is used to complete the returned stage.

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

- ## handleAsync

  <U> CompletionStage<U> handleAsync(BiFunction<? super T, Throwable, ? extends U> fn)
  Returns a new CompletionStage that, when `this` stage completes either normally or exceptionally, is executed using `this` stage's default asynchronous execution facility, with `this` stage's result and exception as arguments to the supplied function.

  When `this` stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments, and the function's result is used to complete the returned stage.

  Type Parameters:
  `U` - the function's return type
  Parameters:
  `fn` - the function to use to compute the value of the returned CompletionStage
  Returns:
  the new CompletionStage

- ## handleAsync

  <U> CompletionStage<U> handleAsync(BiFunction<? super T, Throwable, ? extends U> fn, Executor executor)
  Returns a new CompletionStage that, when `this` stage completes either normally or exceptionally, is executed using the supplied executor, with `this` stage's result and exception as arguments to the supplied function.

  When `this` stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments, and the function's result is used to complete the returned stage.

  Type Parameters:
  `U` - the function's return type
  Parameters:
  `fn` - the function to use to compute the value of the returned CompletionStage
  `executor` - the executor to use for asynchronous execution
  Returns:
  the new CompletionStage

- ## whenComplete

  CompletionStage<T> whenComplete(BiConsumer<? super T, ? super Throwable> action)
  Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action when `this` stage completes.

When `this` stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments. The returned stage is completed when the action returns.

Unlike method `handle`, method `whenComplete` is not designed to translate completion outcomes, so the supplied action should not throw an exception. However, if it does, the following rules apply: if `this` stage completed normally but the supplied action throws an exception, then the returned stage completes exceptionally with the supplied action's exception. Or, if `this` stage completed exceptionally and the supplied action throws an exception, then the returned stage completes exceptionally with `this` stage's exception.

Parameters:
`action` - the action to perform
Returns:
the new CompletionStage

- ### whenCompleteAsync

CompletionStage<T> whenCompleteAsync(BiConsumer<? super T, ? super Throwable> action)
Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action using `this` stage's default asynchronous execution facility when `this` stage completes.

When `this` stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments. The returned stage is completed when the action returns.

Unlike method `handleAsync`, method `whenCompleteAsync` is not designed to translate completion outcomes, so the supplied action should not throw an exception. However, if it does, the following rules apply: If `this` stage completed normally but the supplied action throws an exception, then the returned stage completes exceptionally with the supplied action's exception. Or, if `this` stage completed exceptionally and the supplied action throws an exception, then the returned stage completes exceptionally with `this` stage's exception.

Parameters:
`action` - the action to perform
Returns:
the new CompletionStage

- ## whenCompleteAsync

  CompletionStage<T> whenCompleteAsync(BiConsumer<? super T, ? super Throwable> action, Executor executor)
  Returns a new CompletionStage with the same result or exception as `this` stage, that executes the given action using the supplied Executor when `this` stage completes.

  When `this` stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of `this` stage as arguments. The returned stage is completed when the action returns.

  Unlike method `handleAsync`, method `whenCompleteAsync` is not designed to translate completion outcomes, so the supplied action should not throw an exception. However, if it does, the following rules apply: If `this` stage completed normally but the supplied action throws an exception, then the returned stage completes exceptionally with the supplied action's exception. Or, if `this` stage completed exceptionally and the supplied action throws an exception, then the returned stage completes exceptionally with `this` stage's exception.

  Parameters:
  `action` - the action to perform
  `executor` - the executor to use for asynchronous execution
  Returns:
  the new CompletionStage
- ## exceptionally

  CompletionStage<T> exceptionally(Function<Throwable, ? extends T> fn)
  Returns a new CompletionStage that, when `this` stage completes exceptionally, is executed with `this` stage's exception as the argument to the supplied function. Otherwise, if `this` stage completes normally, then the returned stage also completes normally with the same value.
  Parameters:
  `fn` - the function to use to compute the value of the returned CompletionStage if `this` CompletionStage completed exceptionally
  Returns:
  the new CompletionStage
- ## exceptionallyAsync

  default CompletionStage<T> exceptionallyAsync(Function<Throwable, ? extends T> fn)
  Returns a new CompletionStage that, when `this` stage completes exceptionally, is executed with `this` stage's exception as the argument to the supplied function, using `this` stage's default

asynchronous execution facility. Otherwise, if `this` stage completes normally, then the returned stage also completes normally with the same value.
Implementation Requirements:
The default implementation invokes handle(BiFunction), relaying to handleAsync(BiFunction) on exception, then thenCompose(Function) for result.
Parameters:
`fn` - the function to use to compute the value of the returned CompletionStage if `this` CompletionStage completed exceptionally
Returns:
the new CompletionStage
Since:
12

- ### exceptionallyAsync

default CompletionStage<T> exceptionallyAsync (Function<Throwable, ? extends T> fn, Executor executor)
Returns a new CompletionStage that, when `this` stage completes exceptionally, is executed with `this` stage's exception as the argument to the supplied function, using the supplied Executor. Otherwise, if `this` stage completes normally, then the returned stage also completes normally with the same value.
Implementation Requirements:
The default implementation invokes handle(BiFunction), relaying to handleAsync(BiFunction) on exception, then thenCompose(Function) for result.
Parameters:
`fn` - the function to use to compute the value of the returned CompletionStage if `this` CompletionStage completed exceptionally
`executor` - the executor to use for asynchronous execution
Returns:
the new CompletionStage
Since:
12

- ### exceptionallyCompose

default CompletionStage<T> exceptionallyCompose (Function<Throwable, ? extends CompletionStage<T>> fn)
Returns a new CompletionStage that, when `this` stage completes exceptionally, is composed using the results of the supplied function applied to `this` stage's exception.
Implementation Requirements:
The default implementation invokes handle(BiFunction), invoking the given function on exception, then thenCompose(Function) for result.
Parameters:

`fn` - the function to use to compute the returned CompletionStage if `this` CompletionStage completed exceptionally

Returns:

the new CompletionStage

Since:

12

- ## exceptionallyComposeAsync

  default CompletionStage<T> exceptionallyComposeAsync (Function<Throwable, ? extends CompletionStage<T>> fn)
  Returns a new CompletionStage that, when `this` stage completes exceptionally, is composed using the results of the supplied function applied to `this` stage's exception, using `this` stage's default asynchronous execution facility.

  Implementation Requirements:

  The default implementation invokes `handle(BiFunction)`, relaying to `handleAsync(BiFunction)` on exception, then `thenCompose(Function)` for result.

  Parameters:

  `fn` - the function to use to compute the returned CompletionStage if `this` CompletionStage completed exceptionally

  Returns:

  the new CompletionStage

  Since:

  12

- ## exceptionallyComposeAsync

  default CompletionStage<T> exceptionallyComposeAsync (Function<Throwable, ? extends CompletionStage<T>> fn, Executor executor)
  Returns a new CompletionStage that, when `this` stage completes exceptionally, is composed using the results of the supplied function applied to `this` stage's exception, using the supplied Executor.

  Implementation Requirements:

  The default implementation invokes `handle(BiFunction)`, relaying to `handleAsync(BiFunction)` on exception, then `thenCompose(Function)` for result.

  Parameters:

  `fn` - the function to use to compute the returned CompletionStage if `this` CompletionStage completed exceptionally

  `executor` - the executor to use for asynchronous execution

  Returns:

  the new CompletionStage

  Since:

  12

toCompletableFuture

CompletableFuture<T> toCompletableFuture()
Returns a `CompletableFuture` maintaining the same completion properties as `this` stage. If `this` stage is already a CompletableFuture, method `toCompletableFuture` may return `this` stage itself. Otherwise, invocation may be equivalent in effect to `thenApply(x -> x)`, but returning an instance of type `CompletableFuture`.
Returns:
the CompletableFuture

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual
overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.